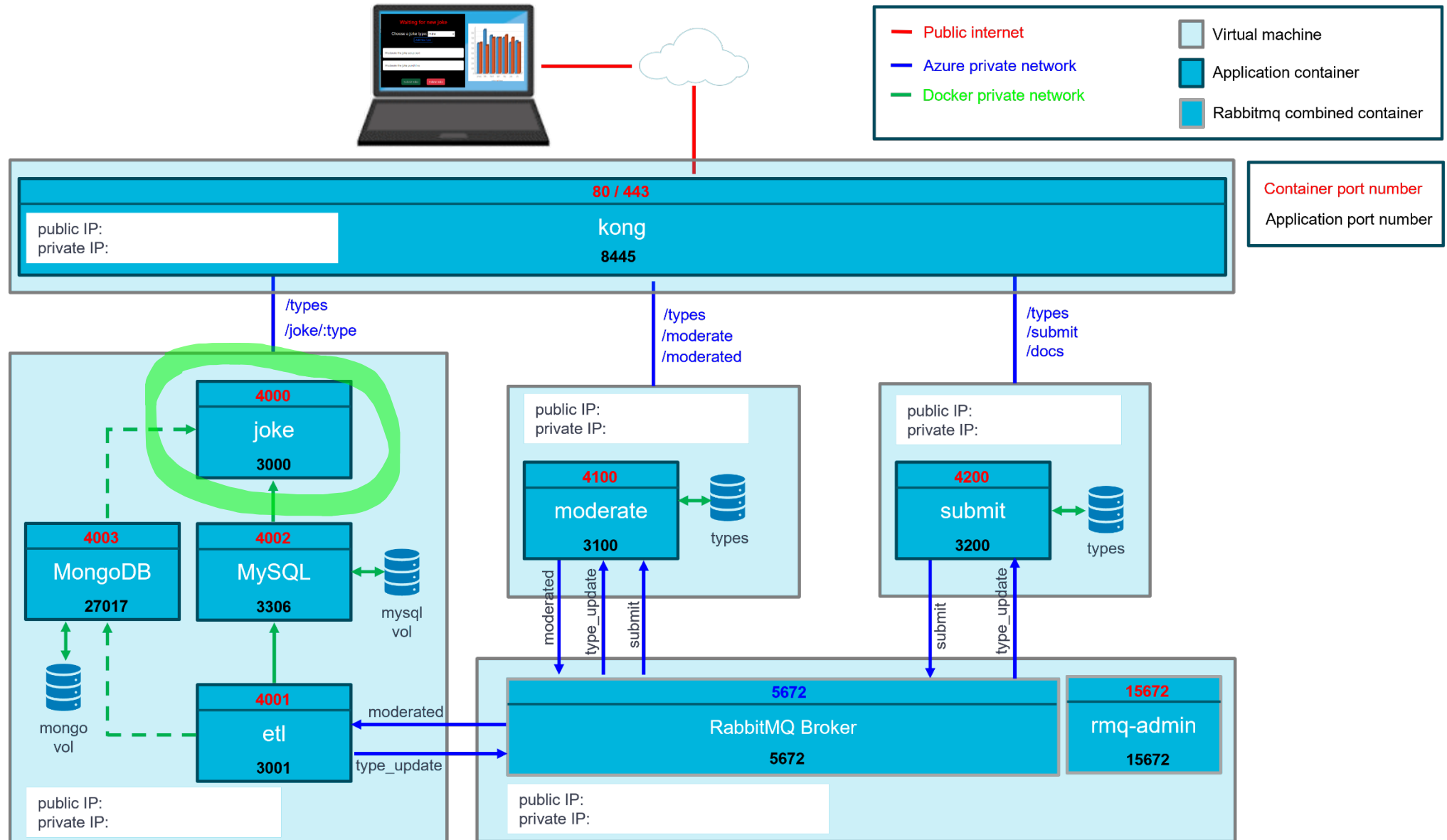


Lab 2: API Server



Practical Exercise: Create a NodeJS API backend

Based on the demos, particularly the provided pricelistBackend example, create a web API to serve up jokes in JSON format. The API will return jokes randomly selected from the database unless a specific joke is requested.

The API has four endpoints: GET /jokes/types, GET /jokes and GET jokes/{id}, POST /joke

GET /jokes/types This returns a unique set of *joke types* from the provided list of jokes in **jokes.json**.

e.g., dad, programming etc

GET /jokes?type={type}&limit={limit} This returns *limit* jokes of type *type*.

If the *type* query parameter is missing, return any type.

If the *limit* query parameter is missing, return one joke

e.g. **GET /jokes?type=dad** returns one dad joke randomly selected from the available list

e.g. **GET /jokes?type=programming&limit=10** returns 10 randomly selected programming jokes

Note: lists should be returned as an array even if only one joke is returned to keep the response consistent. e.g.

```
[ { "id": 28, ... } ]
```

```
[ { "id": 28, ... }, { "id": 32, ... } ]
```

 Obviously ... means including type, setup and punchline

GET /jokes/{id} returns a specific joke based on its ID

e.g., **GET /jokes/28** returns

```
{
  "id": 28,
  "type": "programming",
  "setup": "To understand what recursion is...",
  "punchline": "You must first understand what recursion is"
}
```

POST /jokes submit a new joke to your backend. Still plural as the resource is the referenced collection

Submit the joke as json with an id not already in the array. Prove it works by requesting it by ID

If the user requests more jokes than exists, all the available jokes are returned without an error

If a joke is not found - e.g. there are none of that type or the id is not in the data set, return 404 and an empty array

In the next Lab, you will integrate your frontend with this backend